📊 Práctica de Captura y Visualización de Datos MQTT

🎯 Portada

Asignatura: Sistemas Operativos
Práctica: Captura automática de datos MQTT y visualización en tiempo real
Autor: Martin Penalva
Fecha: Marzo 2026
Objetivo: Desarrollo de un sistema completo de captura, procesamiento y visualización de datos de sensores mediante protocolo MQTT

📖 Introducción y Contextualización del Problema

Contexto del Problema

En el ámbito de los **Sistemas Operativos** y la **gestión de procesos en Linux**, surge la necesidad de monitorear datos de sensores en tiempo real mediante el protocolo **MQTT** (Message Queuing Telemetry Transport). Este protocolo es ampliamente utilizado en **IoT** (Internet of Things) para la comunicación ligera y eficiente entre dispositivos.

Objetivos Técnicos

1. **Captura automática** de datos de múltiples sensores mediante MQTT
2. **Control de procesos** en Linux (PID, señales, gestión de recursos)
3. **Procesamiento de datos** en tiempo real con Python
4. **Visualización** tanto en terminal (ASCII) como gráficamente (PNG)
5. **Combinación de datos** de múltiples sensores en una sola visualización

Desafíos Técnicos

- **Gestión de procesos concurrentes** (MQTT subscribe + análisis)
- **Procesamiento de datos JSON** en tiempo real
- **Control de recursos** y terminación elegante de procesos
- **Visualización escalable** que no rompa la terminal
- **Combinación de múltiples fuentes** de datos

🔧 Explicación Detallada del Script Bash

Estructura General

```bash
#!/bin/bash
set -e  # Salir inmediatamente si hay errores
```

1. Configuración Inicial

```
read -p "Tiempo de captura (segundos): " tiempo
num_capturas=3
espera_entre_capturas=20
```

- **Validación de entrada:** Verifica que el tiempo sea un número válido
- **Variables de control:** Número de capturas y tiempo entre ellas
- **Interacción con usuario:** Solicita parámetros dinámicamente

2. Bucle Principal de Capturas

```
for captura in {1..3}; do
    echo " CAPTURA $captura de $num_capturas"
    # ... lógica de captura ...
done
```

- **Iteración controlada:** Ejecuta exactamente 3 capturas
- **Progreso visual:** Muestra el estado actual
- **Gestión de estado:** Controla el flujo completo

3. Gestión de Archivos

```
./mqtt_subscribe_emqx_linux > mqtt_capture_$captura.log 2>&1 &
```

- **Redirección de salida:** `>` para crear/reescribir archivos
- **Captura de errores:** `2>&1` para incluir stderr
- **Archivos únicos:** `mqtt_capture_1.log`, `mqtt_capture_2.log`, `mqtt_capture_3.log`

4. Control de Tiempo y Espera

```
echo "[3] Esperando $tiempo segundos..."
sleep "$tiempo"
```

- **Pausa controlada:** Espera exactamente el tiempo especificado
- **Feedback visual:** Informa al usuario del progreso
- **Precisión temporal:** Utiliza `sleep` de Unix

🔍 Explicación Detallada del Script Python

1. Importación y Configuración

```python
import json
import os
import matplotlib.pyplot as plt
import numpy as np
from collections import defaultdict
```

- **json:** Para procesar payloads MQTT
- **matplotlib:** Para generación de gráficas PNG
- **collections.defaultdict:** Para manejo eficiente de datos múltiples
- **numpy:** Para cálculos numéricos eficientes

2. Procesamiento de Logs MQTT

```python
with open(log_file, "r") as f:
    lines = f.readlines()

for line in lines:
    line = line.strip()

    if "Topic:" in line:
        if "sensor/data/sen55" in line:
            current_topic = "sensor/data/sen55"
        elif "sensor/data/gas_sensor" in line:
            current_topic = "sensor/data/gas_sensor"
```

- **Lectura secuencial:** Procesa línea por línea
- **Detección de topics:** Identifica sensores específicos
- **Estado persistente:** Mantiene el topic actual entre líneas

3. Extracción de Datos JSON

```python
if "Payload:" in line and current_topic:
    try:
        payload = line.split("Payload:")[1].strip()
        data = json.loads(payload)

        if isinstance(data, dict):
            for key, value in data.items():
                if isinstance(value, (int, float)):
                    combined_values.append(value)
    except:
        pass
```

- **Parsing robusto:** Maneja errores de JSON gracefully
- **Extracción inteligente:** Procesa diccionarios anidados
- **Filtrado de tipos:** Solo acepta valores numéricos

- **Combinación de sensores:** Une datos de múltiples fuentes

4. Visualización ASCII "Montaña Rusa"

```
# Limitar ancho para que no se rompa la terminal
max_width = 60
if len(values) > max_width:
    step_x = len(values) / max_width
    values = [values[int(i * step_x)] for i in range(max_width)]

height = 20  # altura del gráfico
grid = [["  " for _ in range(len(values))] for _ in range(height + 1)]

for x, v in enumerate(values):
    level = int((v - y_min) / step_y)
    level = min(max(level, 0), height)
    grid[height - level][x] = "* "
```

- **Adaptación automática:** Ajusta el ancho a la terminal
- **Submuestreo inteligente:** Mantiene la forma visual
- **Matriz eficiente:** Representación 2D optimizada
- **Escalado automático:** Ajusta valores al rango visual

5. Generación de Gráficas PNG

```
# Gráfica 1: 20 segundos
samples_20s = total_samples // 3
data_20s = combined_values[:samples_20s]

plt.figure(figsize=(12,6))
plt.plot(data_20s, linewidth=2)
plt.title("Todos los datos recogidos - 20 segundos")
plt.xlabel("Índice")
plt.ylabel("Valor")
plt.grid(True)
plt.savefig("plots/dos_sensores_reales_20s.png")
plt.close()
```

- **División temporal:** 20s, 40s, 60s acumulativos
- **Estilo profesional:** Títulos, etiquetas, grid
- **Alta calidad:** Guardado en PNG con buena resolución
- **Memoria eficiente:** Cierra figuras para liberar recursos

---

📸 Capturas de Pantalla de Ejecución

1. Inicio del Script

```
================================
 Captura automática MQTT
================================
Tiempo de captura (segundos): 10
================================
 CAPTURA 1 de 3
================================
[1] Iniciando proceso MQTT...
[2] PID del proceso: 1234
Proceso activo correctamente.
[3] Esperando 10 segundos...
```

Análisis: El sistema solicita el tiempo de captura, inicia el proceso MQTT y muestra el PID asignado por el sistema operativo.

2. Captura de Datos

```
[4] Finalizando proceso con SIGINT...
[5] Proceso finalizado correctamente.
[6] Análisis Python completado para captura 1.
```

Análisis: El proceso se termina elegantemente usando señales Unix (SIGINT → SIGTERM → SIGKILL si es necesario).

3. Visualización ASCII

```
=== 2 SENSORES JUNTOS (ASCII) - 20 segundos ===
muestras=32 | y_min=1 | y_max=494

 494 |             *                       *                       *
 432 |
 370 |
 309 |
 247 |
 185 |                       *                       *
 124 |
  62 |                     *                       *
   1 | * * * * * *   * * *     * * * * * *   * * *     * * * * * *   *
     +-------------------------------------------------------------------------------
--------------------
       0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5  6  7  8  9  0  1  2  3  4
5  6  7  8  9  0  1
```

Análisis: Visualización "Montaña Rusa" mostrando:

- **Picos máximos:** hasta 494 unidades
- **Valores mínimos:** hasta 1 unidad

- **32 muestras** procesadas
- **Patrones repetitivos** de los sensores combinados
- **Distribución clara** de valores altos y bajos

4. Generación de Gráficas

```
✓ 3 gráficas con los 2 sensores reales generadas:
  - plots/dos_sensores_reales_20s.png (primer tercio)
  - plots/dos_sensores_reales_40s.png (dos tercios)
  - plots/dos_sensores_reales_60s.png (todos los datos)
  - Total de datos combinados: 3144 valores
```

Análisis: Sistema genera 3 visualizaciones acumulativas mostrando la evolución temporal de los datos combinados.

🔄 Explicación del Control de Procesos (PID y Señales)

1. Identificación de Procesos

```
./mqtt_subscribe_emqx_linux > mqtt_capture_$captura.log 2>&1 &
PID=$!
echo "[2] PID del proceso: $PID"
```

- **Ejecución en background:** `&` libera la terminal
- **Captura de PID:** `$!` obtiene el ID del último proceso
- **Identificación única:** Cada captura tiene su propio PID

2. Verificación de Estado

```
if kill -0 $PID 2>/dev/null; then
    echo "Proceso activo correctamente."
else
    echo "Error: el proceso no se inició."
    exit 1
fi
```

- `kill -0`**:** Señal nula para verificar si el proceso existe
- **Redirección de errores:** `2>/dev/null` silencia mensajes
- **Validación inmediata:** Confirma que el proceso está corriendo

3. Terminación Elegante (Graceful Shutdown)

```
echo "[4] Finalizando proceso con SIGINT..."
kill -SIGINT $PID 2>/dev/null
sleep 2

if kill -0 $PID 2>/dev/null; then
    echo "Proceso sigue activo → enviando SIGTERM..."
    kill -SIGTERM $PID
    sleep 2
fi

if kill -0 $PID 2>/dev/null; then
    echo "Proceso no responde → enviando SIGKILL..."
    kill -SIGKILL $PID
fi
```

Jerarquía de Señales Unix:

1. **SIGINT (2):** Interrupción desde teclado (Ctrl+C)
    - Permite al proceso limpiar recursos
    - Guarda datos pendientes
    - Cierra conexiones network
2. **SIGTERM (15):** Terminación amigable
    - Solicita terminación voluntaria
    - Permite guardar estado
    - Cierra archivos abiertos
3. **SIGKILL (9):** Terminación forzada
    - No puede ser ignorada
    - Termina inmediatamente
    - Último recurso

4. Gestión de Recursos

```
echo "[5] Proceso finalizado correctamente."
```

- **Confirmación de estado:** Verifica terminación exitosa
- **Limpieza de recursos:** Asegura que no hay procesos zombies
- **Control de flujo:** Permite continuar con siguiente captura

5. Concurrencia y Sincronización

```
if [ $captura -lt $num_capturas ]; then
    echo "[7] Esperando $espera_entre_capturas segundos..."
```

```
    sleep "$espera_entre_capturas"
fi
```

- **Control de concurrencia:** Evita solapamiento de capturas
- **Sincronización temporal:** Espera entre capturas
- **Gestión de estado:** Controla el flujo completo

🎯 Conclusión

Este sistema demuestra un dominio completo de:

- **Gestión de procesos en Linux** con control de PID y señales
- **Programación Bash** para automatización y control
- **Procesamiento de datos** con Python en tiempo real
- **Visualización eficiente** tanto en terminal como gráficamente
- **Integración de sistemas** MQTT + Bash + Python

La práctica combina conceptos fundamentales de Sistemas Operativos con aplicaciones prácticas de IoT, resultando en una solución robusta y escalable para monitoreo de sensores en tiempo real.

📁 **Archivos Generados:**

- `mqtt_capture_1.log`, `mqtt_capture_2.log`, `mqtt_capture_3.log`
- `plots/dos_sensores_reales_20s.png`
- `plots/dos_sensores_reales_40s.png`
- `plots/dos_sensores_reales_60s.png`

🔧 **Tecnologías Utilizadas:**

- **Bash:** Automatización y control de procesos
- **Python:** Procesamiento de datos y visualización
- **MQTT:** Protocolo de comunicación IoT
- **Matplotlib:** Generación de gráficas
- **JSON:** Formato de intercambio de datos